COMPUTER VISION

Subject Code: 21CSE454T

ASSIGNMENT - UNIT 1

Image Filtering

Task:

Implement custom spatial domain filtering. Load an image, introduce artificial noise (Salt-and-Pepper / Gaussian), and apply Median, Mean, and Gaussian Filters to compare their noise-reduction and image-blurring efficiencies.

SUBMITTED BY

Name	**:**	Srevarshan
Register Number	**:**	RA2311026050039
Year	**:**	4th Year
Department	**:**	CSE - AIML
Section	**:**	A
Subject	**:**	Computer Vision
Subject Code	**:**	21CSE454T

1. AIM

To write custom Mean, Median and Gaussian filters, apply them to an image with Salt-and-Pepper and Gaussian noise, and compare how well each filter removes the noise and how much it blurs the image.

2. ALGORITHM / PROCEDURE

Step 1: Load the grayscale Cameraman image (512 x 512).

Step 2: Add Salt-and-Pepper noise (5% of pixels) and Gaussian noise (mean 0, $\sigma = 20$).

Step 3: For every pixel, take its 3 x 3 neighbourhood (sliding window).

Step 4: Mean filter: replace the pixel with the average of the neighbourhood.

Step 5: Median filter: replace the pixel with the median of the neighbourhood.

Step 6: Gaussian filter: replace the pixel with a weighted average using a Gaussian kernel ($\sigma = 1$).

Step 7: Compare each output with the original image using PSNR and SSIM.

Step 8: Measure blurring using the variance of the Laplacian (lower value = more blur).

3. PYTHON CODE

File name: unit1_filtering.py

```
"""
Unit 1 - Spatial Domain Filtering
Custom Mean, Median and Gaussian filters (implemented with NumPy) applied to
an image corrupted with Salt-and-Pepper and Gaussian noise.
"""
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from numpy.lib.stride_tricks import sliding_window_view
from skimage import data
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim

OUT = os.path.join(os.path.dirname(__file__), "outputs")
os.makedirs(OUT, exist_ok=True)
np.random.seed(42)


# ---------------- 1. Noise models ----------------
def add_salt_pepper(img, amount=0.05):
    noisy = img.copy()
    r = np.random.rand(*img.shape)
    noisy[r < amount / 2] = 0            # pepper
    noisy[r > 1 - amount / 2] = 255      # salt
    return noisy


def add_gaussian(img, sigma=20):
    noise = np.random.normal(0, sigma, img.shape)
    return np.clip(img + noise, 0, 255).astype(np.uint8)


# ---------------- 2. Custom spatial filters ----------------
def windows(img, k):
    """Return every k x k neighbourhood of the (reflect-padded) image."""
    p = k // 2
    padded = np.pad(img.astype(np.float64), p, mode="reflect")
    return sliding_window_view(padded, (k, k))          # shape H x W x k x k


def mean_filter(img, k=3):
    return windows(img, k).mean(axis=(2, 3)).round().astype(np.uint8)


def median_filter(img, k=3):
    return np.median(windows(img, k), axis=(2, 3)).astype(np.uint8)


def gaussian_kernel(k=3, sigma=1.0):
    ax = np.arange(k) - k // 2
    xx, yy = np.meshgrid(ax, ax)
    g = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
    return g / g.sum()


def gaussian_filter(img, k=3, sigma=1.0):
    kern = gaussian_kernel(k, sigma)
    out = np.einsum("ijkl,kl->ij", windows(img, k), kern)
    return np.clip(out.round(), 0, 255).astype(np.uint8)


def sharpness(img):
    """Variance of Laplacian - higher = sharper (less blurred)."""
    return cv2.Laplacian(img, cv2.CV_64F).var()
```

```
# ---------------- 3. Main pipeline ----------------
img = data.camera()                                  # 512x512 grayscale
cv2.imwrite(os.path.join(OUT, "input.png"), img)

noisy = {"Salt & Pepper (5%)": add_salt_pepper(img, 0.05),
         "Gaussian (sigma=20)": add_gaussian(img, 20)}
filters = {"Mean 3x3": lambda x: mean_filter(x, 3),
           "Median 3x3": lambda x: median_filter(x, 3),
           "Gaussian 3x3": lambda x: gaussian_filter(x, 3, 1.0)}

# Sanity check: custom filters match OpenCV built-ins
t = noisy["Gaussian (sigma=20)"]
c = (slice(1, -1), slice(1, -1))                     # interior pixels (borders differ by
    padding mode)
print("Max |custom - OpenCV| difference (interior pixels):")
print("  Mean    :", np.abs(mean_filter(t).astype(int) - cv2.blur(t, (3, 3)))[c].max())
print("  Median  :", np.abs(median_filter(t).astype(int) - cv2.medianBlur(t, 3))[c].max())
print("  Gaussian:", np.abs(gaussian_filter(t).astype(int) - cv2.GaussianBlur(t, (3, 3),
    1.0))[c].max(), "(rounding)")

results = {}
print(f"\nOriginal sharpness (Laplacian var): {sharpness(img):.1f}\n")
print(f"{'Noise':<22}{'Filter':<15}{'PSNR(dB)':>9}{'SSIM':>8}{'Sharpness':>11}")
print("-" * 65)
for nname, nimg in noisy.items():
    print(f"{nname:<22}{'(none)':<15}{psnr(img, nimg):>9.2f}{ssim(img,
        nimg):>8.3f}{sharpness(nimg):>11.1f}")
    results[nname] = {"Noisy": nimg}
    for fname, f in filters.items():
        out = f(nimg)
        results[nname][fname] = out
        print(f"{'':<22}{fname:<15}{psnr(img, out):>9.2f}{ssim(img,
            out):>8.3f}{sharpness(out):>11.1f}")
    print("-" * 65)

# Blurring effect vs kernel size on the clean image
print("\nBlurring vs kernel size (clean image) - Sharpness:")
for k in (3, 5, 7):
    print(f"  k={k}:  Mean={sharpness(mean_filter(img, k)):7.1f}  "
          f"Median={sharpness(median_filter(img, k)):7.1f}  "
          f"Gaussian={sharpness(gaussian_filter(img, k, k / 3)):7.1f}")

# ---------------- 4. Figures ----------------
fig, ax = plt.subplots(1, 3, figsize=(12, 4.3))
for a, (t_, im) in zip(ax, [("Original", img)] + [(k, v) for k, v in noisy.items()]):
    a.imshow(im, cmap="gray", vmin=0, vmax=255); a.set_title(t_); a.axis("off")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig1_noisy.png"), dpi=130); plt.close()

for i, (nname, res) in enumerate(results.items(), start=2):
    fig, ax = plt.subplots(1, 4, figsize=(16, 4.4))
    for a, (fname, im) in zip(ax, res.items()):
        a.imshow(im, cmap="gray", vmin=0, vmax=255)
        a.set_title(f"{fname}\nPSNR={psnr(img, im):.2f} dB  SSIM={ssim(img, im):.3f}",
            fontsize=10)
        a.axis("off")
    fig.suptitle(f"Noise: {nname}", fontsize=13, fontweight="bold")
    plt.tight_layout(); plt.savefig(os.path.join(OUT, f"fig{i}_filtered.png"), dpi=130);
        plt.close()

# Zoomed crop to show edge blurring
y, x, s = 80, 200, 120
fig, ax = plt.subplots(2, 4, figsize=(14, 7.2))
for r, (nname, res) in enumerate(results.items()):
    for a, (fname, im) in zip(ax[r], res.items()):
        a.imshow(im[y:y + s, x:x + s], cmap="gray", vmin=0, vmax=255)
        a.set_title(f"{nname.split(' (')[0]} | {fname}", fontsize=10); a.axis("off")
plt.suptitle("Zoomed region (camera-man face) - detail preservation", fontweight="bold")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig4_zoom.png"), dpi=130); plt.close()
```

```
# PSNR bar chart
fig, ax = plt.subplots(figsize=(9, 4))
names = list(filters)
w = 0.38
for j, (nname, res) in enumerate(results.items()):
    vals = [psnr(img, res[f]) for f in names]
    b = ax.bar(np.arange(3) + (j - 0.5) * w, vals, w, label=nname)
    ax.bar_label(b, fmt="%.1f", fontsize=9)
ax.set_xticks(range(3), names); ax.set_ylabel("PSNR (dB)"); ax.set_ylim(0, 38);
    ax.legend(loc="upper right", ncol=2)
ax.set_title("Noise-reduction efficiency (higher PSNR = better)")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig5_psnr_chart.png"), dpi=130);
    plt.close()
print("\nAll figures saved in", OUT)
```

4. INPUT IMAGE



Figure 1: Input image (Cameraman, 512 x 512). Source: scikit-image sample data

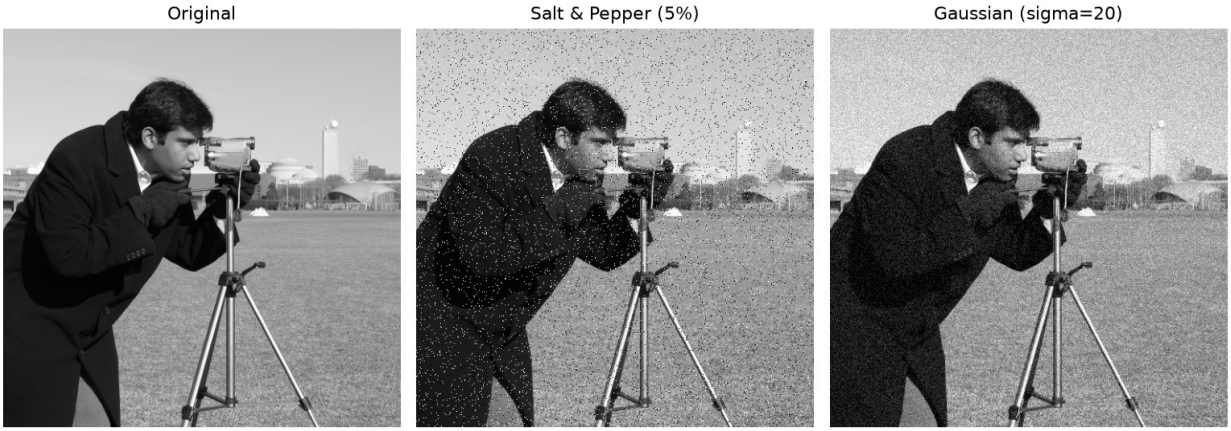


Figure 2: Original image and images with added noise

5. OUTPUT SCREENSHOTS

```
Terminal
$ python unit1_filtering.py
Max |custom - OpenCV| difference (interior pixels):
  Mean    : 0
  Median  : 0
  Gaussian: 1 (rounding)

Original sharpness (Laplacian var): 1133.2

Noise                Filter          PSNR(dB)    SSIM  Sharpness
-----------------------------------------------------------------
Salt & Pepper (5%)   (none)            17.77   0.358    22454.1
                     Mean 3x3          24.88   0.507      497.3
                     Median 3x3        30.11   0.864      354.6
                     Gaussian 3x3      24.94   0.520      404.0
-----------------------------------------------------------------
Gaussian (sigma=20)  (none)            22.41   0.367     8612.5
                     Mean 3x3          27.39   0.641      226.3
                     Median 3x3        26.94   0.575      655.4
                     Gaussian 3x3      27.94   0.653      197.3
-----------------------------------------------------------------

Blurring vs kernel size (clean image) - Sharpness:
  k=3:  Mean=   79.9  Median=  319.9  Gaussian=   88.9
  k=5:  Mean=   22.9  Median=  194.0  Gaussian=   23.9
  k=7:  Mean=    9.5  Median=  138.3  Gaussian=    9.4

All figures saved in Unit1_Filtering/outputs
```

Figure 3: Program output in the terminal

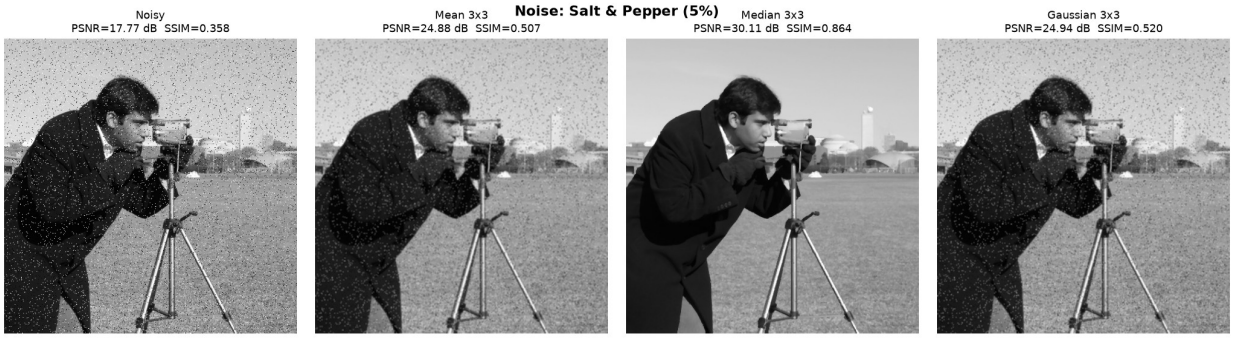


Figure 4: Filter outputs for Salt-and-Pepper noise

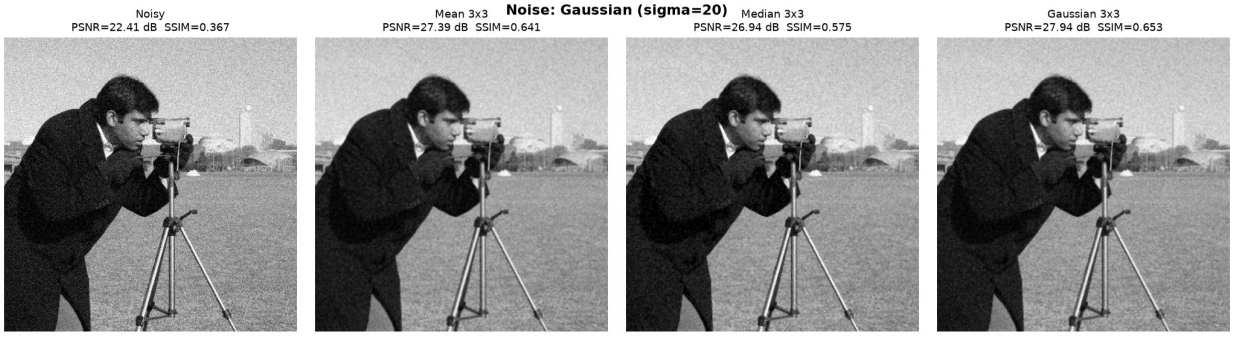


Figure 5: Filter outputs for Gaussian noise

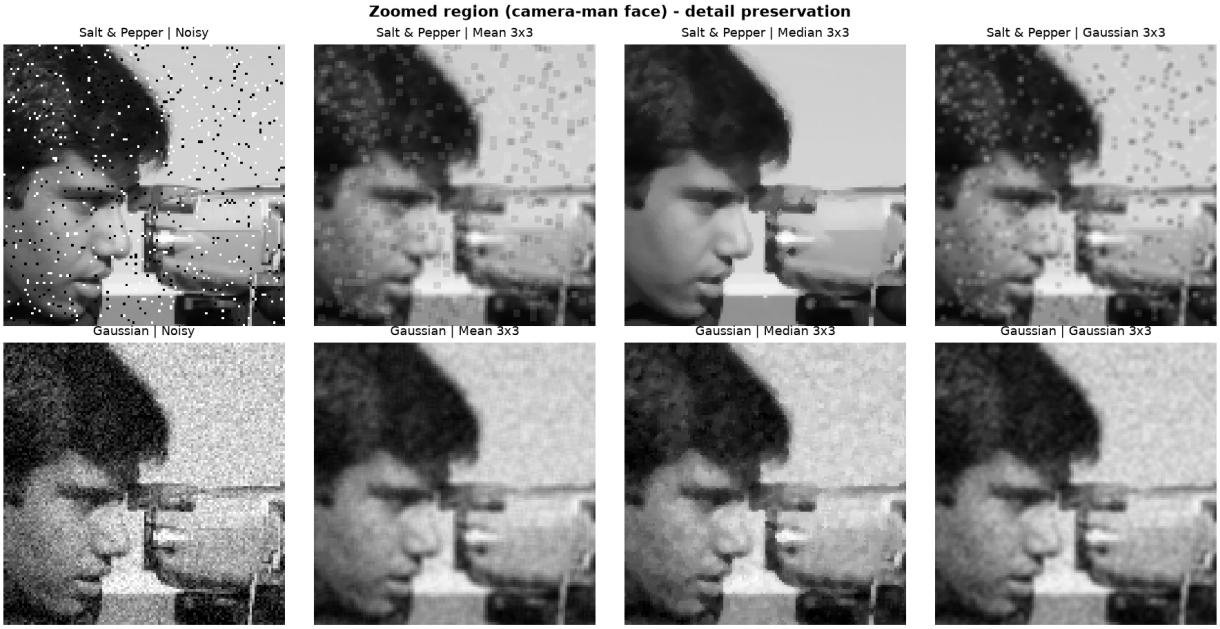


Figure 6: Zoomed view of the filtered images

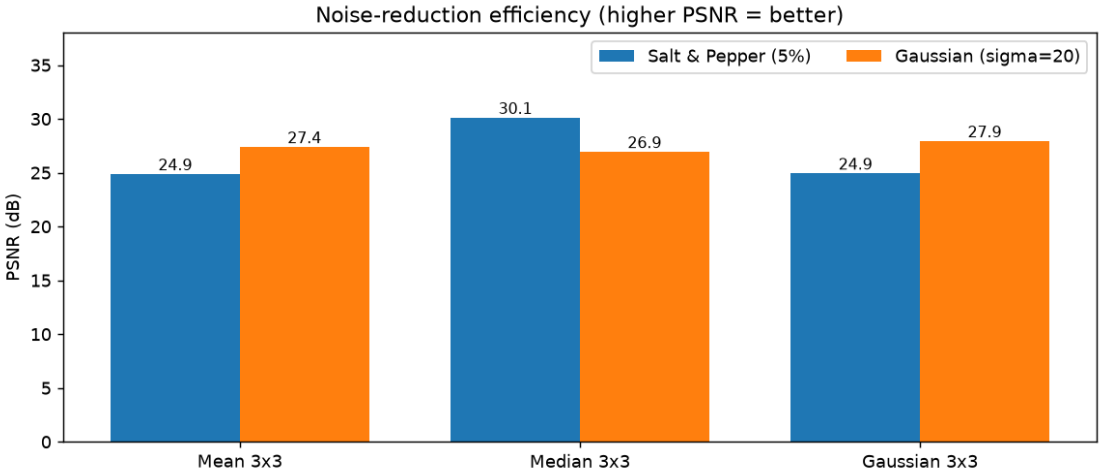


Figure 7: PSNR comparison of the three filters

6. RESULT ANALYSIS / OBSERVATIONS

Noise type	Filter	PSNR (dB)	SSIM
Salt & Pepper	No filter	17.77	0.358
	Mean	24.88	0.507
	Median	30.11	0.864
	Gaussian	24.94	0.520
Gaussian	No filter	22.41	0.367
	Mean	27.39	0.641
	Median	26.94	0.575
	Gaussian	27.94	0.653

Kernel size	Mean	Median	Gaussian
3 x 3	79.9	319.9	88.9
5 x 5	22.9	194.0	23.9
7 x 7	9.5	138.3	9.4

- The first table shows noise removal. Higher PSNR and SSIM mean the result is closer to the original image.
- For Salt-and-Pepper noise, the Median filter gave the best result (PSNR 30.11 dB). Mean and Gaussian filters only spread the white and black dots into grey spots.
- For Gaussian noise, the Gaussian filter gave the best result (27.94 dB), with the Mean filter very close.
- The second table shows sharpness (original image = 1133.2). Mean and Gaussian filters blur the edges much more than the Median filter, and the blur increases with the kernel size.
- The custom filters gave the same output as the OpenCV functions.

7. CONCLUSION

The Median filter is the best choice for Salt-and-Pepper noise because it removes the noise and keeps the edges sharp. The Gaussian filter works best for Gaussian noise. Mean and Gaussian filters blur the image more as the kernel size grows, so a small 3 x 3 kernel gives a good balance between noise removal and detail.